

Summary Notes on Cycle-Jump Techniques

Pruthvi

May 3, 2026

Contents

1	Cojocaru and Karlsson (2006): A Simple Numerical Method of Cycle Jumps for Cyclically Loaded Structures	4
1.1	Full Reference	4
1.2	Main Problem Addressed	4
1.3	Basic Cycle-Jump Idea	4
1.4	General Algorithm	4
1.5	Characteristic Values	5
1.6	Meaning of the Slopes	5
1.7	Control Function for Jump Length	5
1.8	Predicted Slope	6
1.9	Allowed Local Jump Length	6
1.10	Global Jump Length	6
1.11	Heun Extrapolation Formula	6
1.12	Adaptive Number of Full FE Cycles Between Jumps	7
1.13	Why the Control Function Is Necessary	8
1.14	Variables Used in the Control Set	8
1.15	Equivalent Plastic Strain Used for Validation	8
1.16	Computational Efficiency Measure	8
1.17	Error Measures	9
1.18	Cycle-Jump Procedure in Compact Form	9
1.19	Implementation Idea in Abaqus	10
1.20	Key Technical Takeaways	10
1.21	Limitations	10
1.22	Relevance for Thesis Work	11
1.23	One-Sentence Summary	11
2	Nesnas and Saanouni (2000): Cycle-Jumping Scheme for Coupled Damage–Viscoplastic Models	11
2.1	Full Reference	11
2.2	Main Problem Addressed	11
2.3	Main Idea of the Cycle-Jump Method	12
2.4	State Variable as a Function of Cycle Number	12
2.5	Second-Order Taylor Expansion in Cycle Space	12
2.6	First Pseudo-Derivative	13
2.7	First-Order Jump Criterion	13
2.8	Second-Order Jump Criterion	13
2.9	Global Cycle-Jump Size	13
2.10	Why Not Use All State Variables?	14
2.11	Main Variables Used for Jump Control	14
2.12	Cycle-Jump Size Based on ΔL and D	14
2.13	Overall Cycle-Jump Algorithm	15

2.14	Two Versions of the Cycle-Jump Technique	15
2.14.1	CJTV1: Cycle-Jump Technique Version 1	15
2.14.2	CJTV2: Cycle-Jump Technique Version 2	15
2.15	Small Time-Scale Integration	15
2.16	Kinematic Hardening Update	16
2.17	Isotropic Hardening Update	16
2.18	Damage Update	16
2.19	Implicit Unknowns in the Small Time-Scale Algorithm	16
2.20	Extrapolation in CJTV2	17
2.21	Numerical Parameters Used in the Cycle-Jump Method	17
2.22	Important Numerical Observations	17
2.23	Accuracy and Efficiency	18
2.24	Structural-Level Implementation	18
2.25	Key Technical Takeaways	18
2.26	Limitations	19
2.27	Relevance for Thesis Work	19
2.28	One-Sentence Summary	19
3	Li et al. (2025): Adaptive Cycle-Jump Method for Elasto-Plastic Phase-Field Fatigue Crack Propagation	19
3.1	Full Reference	19
3.2	Main Problem Addressed	19
3.3	Main Idea of the Method	20
3.4	Phase-Field Description of Crack	20
3.5	Total Potential Energy	20
3.6	Elastic Energy Split	21
3.7	Plastic Strain Energy Density	21
3.8	Fatigue Degradation Function	21
3.9	Fatigue History Variable	21
3.10	Final Energy Functional with Fatigue	22
3.11	History Field for Crack Irreversibility	22
3.12	Governing Equations	22
3.13	Plastic Constitutive Models	22
3.14	Yield Function	22
3.15	Equivalent Stress for Isotropic Hardening	23
3.16	Plastic Flow Rule	23
3.17	Yield Stress for Linear and Power-Law Hardening	23
3.18	Armstrong–Frederick Model	23
3.19	Finite Element Discretization	24
3.20	Residual Equations	24
3.21	Stiffness Matrices	24
3.22	Cycle-Jump Variable Vector	24
3.23	Why These Variables Are Selected	25
3.24	Evolution Rate of β	25
3.25	Trapezoidal Estimate of Variable Change	25
3.26	Earlier Second-Order Taylor Form	25
3.27	Modified Evolution Rate for Nonlinear Evolution	25
3.28	Expression Using Previous Cycle Values	26
3.29	Adapted Formula Using Three Complete Cycles	26
3.30	Incremental Update Form	26
3.31	Adaptive Cycle-Jump Size	27
3.32	Threshold for Phase-Field Evolution	27
3.33	Correction Coefficient	27
3.34	Final Adaptive Criterion	27

3.35	Element-Wise Jump Size and Global Minimum	27
3.36	Complete Adaptive Explicit Cycle-Jump Algorithm	28
3.37	Behavior of the Adaptive Jump Size	28
3.38	Implementation in Abaqus	28
3.39	Numerical Validation Cases	29
3.40	Main Numerical Results	29
3.41	Paris Law Verification	29
3.42	Key Technical Takeaways	30
3.43	Limitations	30
3.44	Relevance for Thesis Work	30
3.45	One-Sentence Summary	30
4	Cheng, Hu and Kirka (2022): Cycle-Jump Acceleration Method for Crystal Plasticity High-Cycle Fatigue	30
4.1	Full Reference	30
4.2	Main Problem Addressed	31
4.3	Main Idea of the Method	31
4.4	Microstructure-Based Model	31
4.5	Crystal Plasticity Kinematics	31
4.6	Stress Relation	32
4.7	Power-Law Slip Rule	32
4.8	Accumulated Slip	32
4.9	Slip Resistance Hardening	33
4.10	Hall–Petch Grain-Size Dependence	33
4.11	Armstrong–Frederick Backstress Evolution	33
4.12	Internal Variable Set for Cycle Jumping	33
4.13	Core Principle of the Cycle-Jump Method	34
4.14	Two Key Postulations	34
4.15	Why Re-Equilibration Is Needed	34
4.16	Taylor Expansion of Internal Variables	34
4.17	Backward Difference Approximation	35
4.18	Linear Extrapolation Used in the Paper	35
4.19	Adaptive Error Criterion	35
4.20	Allowed Local Jump Size	35
4.21	Global Cycle-Jump Size	35
4.22	Complete Cycle-Jump Algorithm	36
4.23	Known-State-Variable Cycle-Jump Validation	36
4.24	Implementation in Abaqus/Standard	37
4.25	Role of UEXTERNALDB	37
4.26	Role of UMAT	37
4.27	Role of DLOAD and DISP	37
4.28	Effective Cycle Number and Virtual Time	38
4.29	Acceleration Behavior	38
4.30	Validation Cases	38
4.31	Important Numerical Findings	38
4.32	Accuracy Measures	39
4.33	Key Technical Takeaways	39
4.34	Limitations	39
4.35	Relevance for Thesis Work	39
4.36	One-Sentence Summary	40
5	Comparative Summary of Cycle-Jump Techniques	40
6	Next Paper	42

1 Cojocaru and Karlsson (2006): A Simple Numerical Method of Cycle Jumps for Cyclically Loaded Structures

1.1 Full Reference

Cojocaru, D. and Karlsson, A. M. (2006). *A simple numerical method of cycle jumps for cyclically loaded structures*. International Journal of Fatigue, 28, 1677–1689.

1.2 Main Problem Addressed

The paper proposes a cycle-jump method for accelerating finite element simulations of cyclically loaded structures whose state evolves slowly from cycle to cycle.

A direct cycle-by-cycle finite element simulation is expensive because each loading cycle may contain many increments and nonlinear iterations. Therefore, simulating all cycles up to failure or long-term evolution may be computationally infeasible.

The main idea is:

Compute a few complete finite element cycles, identify the long-term trend of the evolving variables, extrapolate the state over many skipped cycles, and restart the finite element simulation from the extrapolated state.

1.3 Basic Cycle-Jump Idea

The total response of a cyclically loaded structure has two parts:

- **Local cyclic response:** the fast oscillation inside each load cycle.
- **Global evolution:** the slow change of the state from one cycle to the next.

The cycle-jump method skips the repeated local cycles and follows the global evolution.

For a state variable $y(t, M)$, where t is time and M is a material point, the method estimates the slow evolution using characteristic values taken at the same point within consecutive cycles.

Examples of possible state variables are:

- displacement components,
- stress components,
- strain components,
- equivalent plastic strain,
- other state-dependent variables.

1.4 General Algorithm

The method can be summarized as follows:

1. Run a small number of full finite element cycles.
2. Extract characteristic values of selected state variables at the same relative location in each cycle.
3. Approximate the global evolution slope of each variable.
4. Use a control function to determine an admissible jump length.
5. Extrapolate the variables using the computed jump length.
6. Impose the extrapolated state as the initial state for the next finite element analysis.
7. Repeat the procedure.

1.5 Characteristic Values

Assume that three consecutive characteristic values of a variable y are available:

$$P_1(t_1, y_1), \quad P_2(t_2, y_2), \quad P_3(t_3, y_3).$$

Here, t_1 is the latest cycle point, while t_2 and t_3 are from previous cycles.
The cycle length is

$$\Delta t_{\text{cycle}} = t_1 - t_2 = t_2 - t_3.$$

The change in the variable over the last two cycles is

$$\Delta y(t_1) = y_1 - y_2,$$

$$\Delta y(t_2) = y_2 - y_3.$$

The corresponding discrete slopes are

$$s_{12}(t_1) = \frac{\Delta y(t_1)}{\Delta t_{\text{cycle}}},$$

$$s_{23}(t_2) = \frac{\Delta y(t_2)}{\Delta t_{\text{cycle}}}.$$

1.6 Meaning of the Slopes

The slope $s_{12}(t_1)$ represents the most recent cycle-to-cycle evolution rate of the variable y .

The slope $s_{23}(t_2)$ represents the previous cycle-to-cycle evolution rate.

If

$$s_{12}(t_1) \approx s_{23}(t_2),$$

then the global evolution is nearly linear and a larger cycle jump is acceptable.

If

$$s_{12}(t_1) \not\approx s_{23}(t_2),$$

then the global evolution is nonlinear and the jump length must be reduced.

1.7 Control Function for Jump Length

The predicted slope after a jump is denoted as

$$s_p(t_1 + \Delta t_{y,\text{jump}}^M).$$

The control condition is

$$\frac{\left| s_p(t_1 + \Delta t_{y,\text{jump}}^M) - s_{12}(t_1) \right|}{|s_{12}(t_1)|} \leq q_y.$$

Here:

- q_y is the user-defined relative error tolerance for variable y ,
- $\Delta t_{y,\text{jump}}^M$ is the locally allowed jump length for variable y at material point M ,
- s_p is the predicted slope after the jump.

1.8 Predicted Slope

The predicted slope after the jump is obtained by linear extrapolation of the slope:

$$s_p(t_1 + \Delta t_{y,\text{jump}}^M) = s_{12}(t_1) + \frac{s_{12}(t_1) - s_{23}(t_2)}{\Delta t_{\text{cycle}}} \Delta t_{y,\text{jump}}^M.$$

This equation predicts how much the evolution rate itself will change after a proposed jump.

1.9 Allowed Local Jump Length

Substituting the predicted slope into the control condition gives the allowed local jump length:

$$\Delta t_{y,\text{jump}}^M = q_y \Delta t_{\text{cycle}} \frac{|s_{12}(t_1)|}{|s_{12}(t_1) - s_{23}(t_2)|}.$$

This is one of the most important equations in the paper.

Interpretation

If the slopes are almost equal,

$$|s_{12}(t_1) - s_{23}(t_2)| \rightarrow 0,$$

then the denominator becomes small and the allowed jump becomes large.

This means the response is nearly linear and a larger cycle jump is safe.

If the slopes differ strongly, then

$$|s_{12}(t_1) - s_{23}(t_2)|$$

becomes large, and the allowed jump becomes small.

This means the response is nonlinear and the method automatically reduces the cycle jump length.

1.10 Global Jump Length

The local jump length is computed for every selected variable and every relevant material point or node.

The global jump length is chosen as the minimum of all local admissible jump lengths:

$$\Delta t_{\text{jump}} = \Delta t_{\text{cycle}} \left\lfloor \frac{\min \left\{ \Delta t_{y,\text{jump}}^M \right\}}{\Delta t_{\text{cycle}}} \right\rfloor.$$

Here, $\lfloor \cdot \rfloor$ is the floor function.

Meaning

The method uses the most restrictive variable or point to decide the global jump length.

This is conservative but important, because one rapidly evolving point can make a large jump inaccurate.

1.11 Heun Extrapolation Formula

After the global jump length is selected, the variable y is extrapolated using a Heun-type integration formula:

$$y(t_1 + \Delta t_{\text{jump}}) = y(t_1) + \frac{1}{2} [s_{12}(t_1) + s_p(t_1 + \Delta t_{\text{jump}})] \Delta t_{\text{jump}}.$$

Substituting the expression for s_p gives

$$y(t_1 + \Delta t_{\text{jump}}) = y(t_1) + s_{12}(t_1)\Delta t_{\text{jump}} + \frac{[s_{12}(t_1) - s_{23}(t_2)] (\Delta t_{\text{jump}})^2}{2\Delta t_{\text{cycle}}}.$$

Interpretation

The first term,

$$y(t_1),$$

is the current value.

The second term,

$$s_{12}(t_1)\Delta t_{\text{jump}},$$

is the linear extrapolation using the latest slope.

The third term,

$$\frac{[s_{12}(t_1) - s_{23}(t_2)] (\Delta t_{\text{jump}})^2}{2\Delta t_{\text{cycle}}},$$

corrects the extrapolation using the change in slope between the last two cycles.

Therefore, the method is more accurate than a simple forward Euler extrapolation.

1.12 Adaptive Number of Full FE Cycles Between Jumps

After each jump, the method does not immediately jump again. It computes some full finite element cycles before the next jump.

The number of full FE cycles after jump j is denoted by $N_{\text{FEA},j}$.

The empirical update rule is

$$N_{\text{FEA},j} = N_{\text{FEA},j-1} + \Delta_+, \quad \text{if } \frac{\Delta t_{\text{jump},j}}{\Delta t_{\text{cycle}}} \leq 1,$$

$$N_{\text{FEA},j} = N_{\text{FEA},j-1} - \Delta_-,$$

if

$$\frac{\Delta t_{\text{jump},j}}{\Delta t_{\text{cycle}}} \geq N_{\text{FEA},j-1}$$

and

$$N_{\text{FEA},j-1} - \Delta_- \geq N_{\text{min}}.$$

Otherwise,

$$N_{\text{FEA},j} = N_{\text{FEA},j-1}.$$

Here:

- Δ_+ increases the number of full FE cycles when jumps become too short,
- Δ_- decreases the number of full FE cycles when jumps are sufficiently long,
- N_{min} is the minimum number of full FE cycles allowed between jumps.

1.13 Why the Control Function Is Necessary

The paper shows that finite element convergence after a cycle jump does not guarantee that the solution is physically correct.

A large uncontrolled jump may converge numerically but still produce wrong stress and plastic strain fields.

Therefore, the control function is essential because it controls:

- the slope change,
- the jump length,
- the balance between accuracy and computational efficiency.

1.14 Variables Used in the Control Set

In the examples of the paper, the control set mainly includes displacement components:

$$u_1, \quad u_2.$$

However, the method is general. The control set may include:

$$\mathcal{C} = \{u_i, \sigma_{ij}, \varepsilon_{ij}, \varepsilon_{eq}^p, \dots\}.$$

The global jump length is then controlled by the most restrictive variable in $\mathcal{C}$.

1.15 Equivalent Plastic Strain Used for Validation

The equivalent plastic strain is used as one of the important quantities to assess the accuracy of the cycle-jump result.

For classical plasticity, the equivalent plastic strain is defined as

$$\varepsilon_{eq}^p = \int d\varepsilon_{eq}^p = \int \left(\frac{2}{3} d\varepsilon_{ij}^p d\varepsilon_{ij}^p \right)^{1/2}.$$

This quantity accumulates with plastic deformation and is useful for checking whether the cycle-jump solution captures progressive inelastic evolution.

1.16 Computational Efficiency Measure

The computational efficiency is measured using

$$R = \frac{L_{\text{jumps}}}{L_{\text{tot}}},$$

where:

- L_{jumps} is the total time or number of cycles covered by jumps,
- L_{tot} is the total time or number of cycles considered in the analysis.

A larger value of R means that more of the simulation was covered by cycle jumps and fewer cycles were computed directly.

1.17 Error Measures

The relative error of a state variable is defined as

$$\delta E = \left| \frac{y_{\text{ref}} - y_{\text{jump}}}{y_{\text{ref}}} \right| \times 100\%.$$

Here:

- y_{ref} is the reference cycle-by-cycle solution,
- y_{jump} is the cycle-jump solution.

The average relative error over all integration points is

$$\overline{\delta E} = \frac{1}{N} \sum_{k=1}^N \delta E_k.$$

The standard deviation of the relative error is

$$s = \sqrt{\frac{1}{N} \sum_{k=1}^N (\delta E_k - \overline{\delta E})^2}.$$

1.18 Cycle-Jump Procedure in Compact Form

1. Select a control set of variables $\mathcal{C}$.
2. Run a few full FE cycles.
3. For each control variable y and material point M , extract

$$y_1, \quad y_2, \quad y_3.$$

4. Compute the slopes

$$s_{12}(t_1) = \frac{y_1 - y_2}{\Delta t_{\text{cycle}}}, \quad s_{23}(t_2) = \frac{y_2 - y_3}{\Delta t_{\text{cycle}}}.$$

5. Compute the locally allowed jump length

$$\Delta t_{y,\text{jump}}^M = q_y \Delta t_{\text{cycle}} \frac{|s_{12}(t_1)|}{|s_{12}(t_1) - s_{23}(t_2)|}.$$

6. Compute the global jump length

$$\Delta t_{\text{jump}} = \Delta t_{\text{cycle}} \left\lfloor \frac{\min \left\{ \Delta t_{y,\text{jump}}^M \right\}}{\Delta t_{\text{cycle}}} \right\rfloor.$$

7. Extrapolate the variable:

$$y(t_1 + \Delta t_{\text{jump}}) = y(t_1) + s_{12}(t_1) \Delta t_{\text{jump}} + \frac{[s_{12}(t_1) - s_{23}(t_2)](\Delta t_{\text{jump}})^2}{2\Delta t_{\text{cycle}}}.$$

8. Impose the extrapolated state in the FE model.
9. Run full FE cycles again.
10. Repeat until the target number of cycles is reached.

1.19 Implementation Idea in Abaqus

The paper uses Abaqus with external scripting and user subroutines.

The implementation logic is:

- Abaqus/CAE and Python scripting are used to generate and manage the model.
- Abaqus solves the complete cycles.
- Results are extracted from the output database.
- Extrapolated states are written to external files.
- User subroutines impose the extrapolated state.

The important subroutines mentioned are:

- SDVINI: initialization of state variables,
- DISP: prescribed extrapolated displacements,
- UEXPAN: imposed stress-free growth strain,
- UMAT: material behavior and insertion of extrapolated stress/strain information.

1.20 Key Technical Takeaways

- The method extrapolates the global evolution of structural variables, not the detailed intra-cycle oscillations.
- The control function is the central contribution of the paper.
- The jump length is automatically shortened when the response becomes nonlinear.
- A converged finite element solution after a jump is not necessarily correct.
- Accuracy must be checked using state-variable comparison with a reference or controlled error measure.
- The method is most suitable for quasi-linear global evolution but can still handle nonlinear regions by reducing or suppressing jumps.

1.21 Limitations

- The method depends on user-defined tolerance values q_y .
- If q_y is too large, the method may take jumps that are too long and lose accuracy.
- If q_y is too small, the method becomes accurate but less efficient.
- The approach requires careful selection of control variables.
- The method does not automatically guarantee correctness only from finite element convergence.

1.22 Relevance for Thesis Work

This paper is important because it gives one of the clearest early finite-element cycle-jump algorithms.

For future UMAT or Abaqus cycle-jump implementation, the most useful ideas are:

- extract cycle-to-cycle evolution of selected variables,
- define a control set,
- compute local allowed jump lengths,
- take the global minimum jump length,
- extrapolate using a Heun-type formula,
- reinsert the extrapolated state into Abaqus,
- continue the finite element simulation after the jump.

1.23 One-Sentence Summary

Cojocaru and Karlsson’s method accelerates cyclic finite element simulations by computing a few real cycles, estimating the global evolution slope of selected state variables, using a control function to choose a safe jump length, and extrapolating the state with a Heun-type formula before continuing the finite element analysis.

2 Nesnas and Saanouni (2000): Cycle-Jumping Scheme for Coupled Damage–Viscoplastic Models

2.1 Full Reference

Nesnas, K. and Saanouni, K. (2000). *A cycle jumping scheme for numerical integration of coupled damage and viscoplastic models for cyclic loading paths*. Revue Européenne des Éléments Finis, 9(8), 865–891.

2.2 Main Problem Addressed

This paper develops a cycle-jumping scheme for the numerical integration of coupled damage and viscoplastic constitutive models under cyclic loading.

The problem is that a fully coupled damage–viscoplastic simulation requires the constitutive equations to be integrated through many loading cycles. This is computationally expensive because:

- the constitutive equations are nonlinear and stiff,
- damage evolves progressively over cycles,
- viscoplasticity and damage are coupled,
- direct cycle-by-cycle integration may require thousands of cycles.

The proposed solution is a **two time-scale integration scheme**:

- **Small time scale**: implicit integration inside each loading cycle.
- **Large time scale**: explicit cycle-jump integration over many cycles.

2.3 Main Idea of the Cycle-Jump Method

The fatigue life is divided into three stages:

1. **Hardening stage:** stress increases from cycle to cycle.
2. **Stabilized stage:** stress redistribution is slow.
3. **Softening stage:** damage grows rapidly and stress decreases until failure.

The stabilized stage is usually the longest part of the fatigue life. Therefore, the method skips some intermediate cycles during this stage.

The central idea is:

Integrate accurately inside a few real cycles, estimate how selected variables evolve from cycle to cycle, jump over several cycles, and extrapolate the material state at the new cycle number.

2.4 State Variable as a Function of Cycle Number

Let T be the period of one loading cycle. A particular instant τ inside each cycle is selected, where

$$0 \leq \tau \leq T.$$

The vector of state variables is then written as a function of cycle number N :

$$\mathbf{y}(N) = \mathbf{y}((N-1)T + \tau).$$

Here:

- $\mathbf{y}$ is the vector of state variables,
- N is the cycle number,
- T is the cycle period,
- τ is the selected point inside the cycle.

The problem becomes the determination of a sequence of values

$$\mathbf{y}(0 \leq N \leq N_R),$$

where N_R is the number of cycles to failure.

2.5 Second-Order Taylor Expansion in Cycle Space

The state variable after a cycle jump ΔN is approximated using a Taylor expansion in cycle space:

$$\mathbf{y}_{N+\Delta N} = \mathbf{y}_N + \dot{\mathbf{y}}_N \Delta N + \ddot{\mathbf{y}}_N \frac{\Delta N^2}{2}.$$

Here:

- $\dot{\mathbf{y}}_N$ is the first pseudo-derivative with respect to cycle number,
- $\ddot{\mathbf{y}}_N$ is the second pseudo-derivative with respect to cycle number,
- ΔN is the number of cycles skipped.

This is not a time derivative inside one loading cycle. It is a derivative with respect to the cycle number.

2.6 First Pseudo-Derivative

The first pseudo-derivative is approximated as

$$\dot{\mathbf{y}}_N = \frac{\partial \mathbf{y}}{\partial N} \approx \frac{\mathbf{y}_{N+1} - \mathbf{y}_N}{1}.$$

So, in simple form,

$$\dot{\mathbf{y}}_N \approx \mathbf{y}_{N+1} - \mathbf{y}_N.$$

This represents the cycle-to-cycle change of the state variable vector.

2.7 First-Order Jump Criterion

The first possibility is to keep only the first-order Taylor term. The jump size is chosen so that the first-order extrapolated change remains small compared with the current variable value:

$$|\dot{\mathbf{y}}_N| \Delta N \leq \eta_1 |\mathbf{y}_N|.$$

Solving for the jump size gives

$$\Delta N = \eta_1 \left| \frac{\mathbf{y}_N}{\dot{\mathbf{y}}_N} \right|.$$

Here, η_1 is an accuracy parameter.

Meaning

If the state variable changes slowly from cycle to cycle, then $|\dot{\mathbf{y}}_N|$ is small and ΔN becomes large.

If the state variable changes rapidly, then $|\dot{\mathbf{y}}_N|$ is large and ΔN becomes small.

2.8 Second-Order Jump Criterion

The second possibility is to use a second-order Taylor development. The second-order term is controlled relative to the first-order term:

$$|\ddot{\mathbf{y}}_N| \frac{\Delta N^2}{2} \leq \eta_2 |\dot{\mathbf{y}}_N| \Delta N.$$

Solving for ΔN gives

$$\Delta N = 2\eta_2 \left| \frac{\dot{\mathbf{y}}_N}{\ddot{\mathbf{y}}_N} \right|.$$

Here, η_2 is another accuracy parameter.

Meaning

This condition controls the curvature of the state-variable evolution.

If the state evolution is almost linear, $\ddot{\mathbf{y}}_N$ is small and a large jump is allowed.

If the state evolution is strongly nonlinear, $\ddot{\mathbf{y}}_N$ is large and the jump size is reduced.

2.9 Global Cycle-Jump Size

The local jump size is evaluated for the chosen variables at each integration point.

The global cycle-jump increment is then selected as the minimum value:

$$\Delta N = \min_i (\Delta N_i).$$

This means the most critical integration point controls the global jump size.

2.10 Why Not Use All State Variables?

The classical cycle-jump technique uses all state variables to estimate ΔN .

However, Nesnas and Saanouni point out that this can be difficult because different state variables:

- evolve at different rates,
- may have very different magnitudes,
- may show different degrees of nonlinearity,
- may give artificially small or unsuitable jump sizes.

Therefore, the paper proposes using only a small number of representative variables that measure the global evolution of the material state.

2.11 Main Variables Used for Jump Control

The paper identifies two important variables for controlling the cycle jump:

$$\Delta L \quad \text{and} \quad D.$$

Here:

- ΔL is an important scalar parameter from the asymptotic integration algorithm,
- D is the damage variable.

The parameter ΔL is sensitive to the nonlinear viscoplastic evolution, especially during hardening and stabilized stages.

The damage variable D is especially important during the softening stage, where damage grows rapidly near failure.

2.12 Cycle-Jump Size Based on ΔL and D

The paper proposes computing two candidate jump sizes:

$$\Delta N_{\Delta L} = \eta \frac{\Delta L}{\dot{\Delta L}},$$

$$\Delta N_D = \eta \frac{D}{\dot{D}}.$$

The final jump size is

$$\Delta N = \min(\Delta N_{\Delta L}, \Delta N_D).$$

Interpretation

The variable ΔL controls the jump during viscoplastic hardening and stabilization.

The variable D controls the jump during the damage-dominated softening stage.

Taking the minimum makes the algorithm conservative.

2.13 Overall Cycle-Jump Algorithm

The cycle-jump algorithm works as follows:

1. After each cycle jump, integrate the constitutive equations over a small number of real cycles.
2. Store the values of the selected jump-control variables.
3. Compute the first and second pseudo-derivatives in cycle space.
4. Estimate the cycle-jump increment ΔN .
5. Extrapolate the state variables over ΔN cycles.
6. Continue the full integration for several cycles after the jump.
7. Repeat until failure.

In the paper, a typical value used for the number of real cycles before jumping is five cycles.

2.14 Two Versions of the Cycle-Jump Technique

The paper introduces two versions of the cycle-jump technique.

2.14.1 CJTV1: Cycle-Jump Technique Version 1

In the first version, all state variables are extrapolated directly using the Taylor expansion:

$$\mathbf{y}_{N+\Delta N} = \mathbf{y}_N + \dot{\mathbf{y}}_N \Delta N + \ddot{\mathbf{y}}_N \frac{\Delta N^2}{2}.$$

This version is straightforward but requires storing and extrapolating all state variables.

2.14.2 CJTV2: Cycle-Jump Technique Version 2

In the second version, only the selected scalar parameters are extrapolated:

$$\Delta L, \quad \Delta p, \quad D.$$

Then, the remaining state variables are reconstructed using the asymptotic integral forms of the constitutive equations.

This reduces the number of variables that must be stored and extrapolated.

2.15 Small Time-Scale Integration

Inside each loading cycle, the paper uses an implicit asymptotic integration algorithm.

The coupled damage–viscoplastic model includes:

- Cauchy stress σ_{ij} ,
- kinematic hardening variables α_{ij}^k ,
- isotropic hardening variable r ,
- damage variable D .

The recursive update of stress has the form

$$\begin{aligned} \sigma_{ij}(t + \Delta t) = & X_{ij}(t + \Delta t) + \frac{\nu E}{(1 + \nu)(1 - 2\nu)} \varepsilon_{kk}(t) \delta_{ij} \\ & + \exp(-\Delta L) \left[\sigma_{ij}(t) - X_{ij}(t) - \frac{\nu E}{(1 + \nu)(1 - 2\nu)} \varepsilon_{kk}(t) \delta_{ij} \right] + \dots \end{aligned}$$

where ΔL appears as a central scalar parameter in the integration algorithm.

2.16 Kinematic Hardening Update

The kinematic hardening variable is updated as

$$\alpha_{ij}^k(t + \Delta t) = \exp(-\Delta G^k) \alpha_{ij}^k(t) + \Delta \varepsilon_{ij}^p \left[\frac{1 - \exp(-\Delta G^k)}{\Delta G^k} \right].$$

The scalar parameter ΔG^k is

$$\Delta G^k = a_k \sqrt{1 - D} \Delta p.$$

2.17 Isotropic Hardening Update

The isotropic hardening variable is updated as

$$r(t + \Delta t) = \exp(-\Delta Q) r(t) + \Delta p \left[\frac{1 - \exp(-\Delta Q)}{\Delta Q} \right].$$

The scalar parameter ΔQ is

$$\Delta Q = b \sqrt{1 - D} \Delta p.$$

2.18 Damage Update

The damage variable can be updated using a first-order approximation:

$$D(t + \Delta t) = D(t) + \Delta D = D(t) + \dot{D}(t) \Delta t.$$

A second-order approximation can also be used:

$$D(t + \Delta t) = D(t) + \Delta D = D(t) + \frac{1}{2} [\dot{D}(t) + \dot{D}(t + \Delta t)] \Delta t.$$

The paper shows that the order of damage extrapolation is important, especially near the final damage-dominated stage.

2.19 Implicit Unknowns in the Small Time-Scale Algorithm

Although the complete constitutive model contains many variables, the asymptotic algorithm reduces the implicit numerical problem to two main scalar unknowns:

$$\Delta L \quad \text{and} \quad \Delta D.$$

These are solved using a Newton–Raphson procedure.

The nonlinear system is written as

$$g_1(\Delta L, \Delta D) = 0,$$

$$g_2(\Delta L, \Delta D) = 0.$$

In compact vector form,

$$g_i(x_j) = 0,$$

where

$$x_1 = \Delta L, \quad x_2 = \Delta D.$$

The Newton linearization is

$$\frac{\partial g_i}{\partial x_j} \delta x_j = -g_i(x_j).$$

The iterative update is

$$x_j^{k+1} = x_j^k + \delta x_j^k.$$

The convergence criteria are

$$\frac{\|\delta x_j^k\|_2}{\|x_j^k\|_2} \leq \varepsilon_1,$$

$$\frac{\|g_j^k\|_2}{\|g_j^1\|_2} \leq \varepsilon_2.$$

2.20 Extrapolation in CJTV2

In CJTV2, after extrapolating

$$\Delta L, \quad \Delta p, \quad D,$$

the remaining state variables are reconstructed.

For example, the stress after the cycle jump is reconstructed using an asymptotic expression of the form

$$\begin{aligned} \sigma_{ij}^{N+\Delta N}(t + \Delta t) = & \frac{2}{3} C \alpha_{ij}^{N+\Delta N}(t + \Delta t) + \frac{\nu E}{(1 + \nu)(1 - 2\nu)} \varepsilon_{kk}(t + \Delta t) \delta_{ij} \\ & + \exp(-\Delta L^{N+\Delta N}) \left[\sigma_{ij}(t) - \frac{2}{3} C \alpha_{ij}(t) - \frac{\nu E}{(1 + \nu)(1 - 2\nu)} \varepsilon_{kk}(t) \delta_{ij} \right] + \dots \end{aligned}$$

This is why CJTV2 stores fewer variables but still reconstructs the complete material state.

2.21 Numerical Parameters Used in the Cycle-Jump Method

The paper introduces several practical parameters:

- τ : selected instant inside the cycle,
- JUMPMIN: minimum number of cycles computed before the next jump,
- η : accuracy parameter for the jump size,
- JUMPMAX: maximum allowed number of jumped cycles.

These parameters control the balance between accuracy and speed.

2.22 Important Numerical Observations

The paper reports the following key observations:

- Large jumps are possible in low-curvature stabilized regions.
- Smaller jumps are automatically selected near failure.
- Damage requires careful extrapolation because it evolves nonlinearly near the end of life.
- Second-order extrapolation of damage gives better fatigue-life prediction than first-order extrapolation.
- CJTV2 is efficient because it extrapolates only selected scalar parameters and reconstructs the remaining variables.

2.23 Accuracy and Efficiency

For proportional loading at Gauss point level, the full calculation gave a reference fatigue life of approximately

$$N_R = 384 \text{ cycles.}$$

Using cycle jumping, the method predicted similar fatigue lives while computing far fewer cycles. For example, with $\eta = 0.1$:

$$N_R \approx 392 \text{ cycles}$$

for CJTV1, and

$$N_R \approx 391 \text{ cycles}$$

for CJTV2.

Thus, both schemes saved about 80% of the actually computed cycles with a small error in fatigue life.

For a long non-proportional loading example, CJTV2 reduced the computational effort by more than 90% while maintaining good agreement with the full calculation.

2.24 Structural-Level Implementation

At structural level, the cycle jump step is calculated at every Gauss integration point.

The global cycle-jump size is

$$\Delta N = \min_{\text{all Gauss points}} (\Delta N_i).$$

This ensures that the most critical material point controls the global jump.

The method was tested on:

- a simplified three-bar structure,
- a plate with a central circular hole.

In both cases, the method reproduced the full cycle-by-cycle solution with significantly reduced computation time.

2.25 Key Technical Takeaways

- This paper uses a two time-scale strategy.
- The small time scale is handled by an implicit asymptotic integration algorithm.
- The large time scale is handled by an explicit cycle-jump algorithm.
- The cycle-jump variable is not necessarily the full state vector.
- The scalar variables ΔL and D are enough to estimate the jump size effectively.
- The global jump size is the minimum jump size over all integration points.
- Damage evolution strongly controls the final softening stage.
- Second-order extrapolation is important for nonlinear damage evolution.

2.26 Limitations

- The method still depends on user-defined parameters such as η , JUMPMIN, and JUMPMAX.
- The accuracy depends on the selected jump-control variables.
- Strongly nonlinear damage evolution near failure requires smaller jumps.
- The presented damage model does not include unilateral damage effects, where microcracks may open or close depending on the loading direction.

2.27 Relevance for Thesis Work

This paper is important because it treats cycle jumping at the constitutive integration level.

Compared with structural cycle-jump methods that extrapolate displacements, stresses, or strains directly, this method focuses on internal material-state evolution.

For future UMAT or constitutive implementation work, the important lessons are:

- separate the intra-cycle integration from the cycle-space extrapolation,
- use robust implicit integration inside the cycle,
- use explicit extrapolation only over cycles,
- choose a small set of physically meaningful jump-control variables,
- reduce the global jump size near damage localization or failure,
- reconstruct the full state after the jump instead of storing every state variable.

2.28 One-Sentence Summary

Nesnas and Saanouni’s method accelerates coupled damage–viscoplastic cyclic simulations by using implicit asymptotic integration inside each cycle and explicit cycle-space extrapolation between cycles, with the jump length controlled mainly by the scalar variables ΔL and damage D .

3 Li et al. (2025): Adaptive Cycle-Jump Method for Elasto-Plastic Phase-Field Fatigue Crack Propagation

3.1 Full Reference

Li, J., Hu, Y., Ao, N., Miao, H., Zhang, X., Kang, G., and Kan, Q. (2025). *An adaptive cycle jump method for elasto-plastic phase field modeling addressing fatigue crack propagation*. Computer Methods in Applied Mechanics and Engineering, 442, 118074.

3.2 Main Problem Addressed

This paper develops an adaptive cycle-jump method for accelerating phase-field simulations of fatigue crack propagation in elasto-plastic materials.

Phase-field fatigue simulations are computationally expensive because:

- the crack is represented by a diffused phase-field variable, requiring a fine mesh near the crack path,
- fatigue crack propagation normally needs cycle-by-cycle computation,
- elasto-plastic crack-tip behavior introduces additional nonlinear constitutive cost,
- the crack propagation life is usually not known in advance.

The proposed method accelerates the simulation by explicitly extrapolating selected variables over skipped cycles and adaptively choosing the cycle-jump size based on phase-field evolution.

3.3 Main Idea of the Method

The key idea is:

Compute three complete cycles, use their results to estimate the evolution trend of important phase-field and plasticity variables, jump over ΔN cycles, and update the variables at cycle $N + \Delta N$.

Unlike purely elastic phase-field fatigue methods, this method also accounts for plastic strain accumulation at the crack tip.

3.4 Phase-Field Description of Crack

The body is described by two primary fields:

$$\mathbf{u} \quad \text{and} \quad \phi.$$

Here:

- $\mathbf{u}$ is the displacement field,
- ϕ is the phase-field crack variable,
- $\phi = 0$ represents intact material,
- $\phi = 1$ represents fully cracked material.

The degradation function is

$$g(\phi) = (1 - \phi)^2 + \kappa,$$

where κ is a small numerical parameter introduced to avoid singularity when the material is fully damaged.

3.5 Total Potential Energy

For an elasto-plastic body, the total potential energy is written as

$$\begin{aligned} \Psi = & \int_{\Omega} [g(\phi)\psi^e(\boldsymbol{\varepsilon}^e) + g(\phi)\psi^p(\boldsymbol{\varepsilon}^p)] dV \\ & + \int_{\Omega} \frac{G_c}{2l} (\phi^2 + l^2 |\nabla \phi|^2) dV - \int_{\Omega} \mathbf{b} \cdot \mathbf{u} dV - \int_{\partial\Omega} \mathbf{t} \cdot \mathbf{u} dS. \end{aligned}$$

Here:

- ψ^e is the elastic strain energy density,
- ψ^p is the plastic strain energy density,
- G_c is the critical energy release rate,
- l is the phase-field length scale,
- $\mathbf{b}$ is the body force,
- $\mathbf{t}$ is the boundary traction.

3.6 Elastic Energy Split

The tensile and compressive parts of the elastic strain energy density are written as

$$\psi_e^\pm = \frac{1}{2} \lambda \langle \text{tr}(\boldsymbol{\varepsilon}^e) \rangle_\pm^2 + \mu \text{tr} [(\boldsymbol{\varepsilon}_e^\pm)^2].$$

Here:

- λ and μ are Lamé constants,
- $\langle x \rangle_\pm = \frac{x \pm |x|}{2}$ is the Macaulay bracket.

The energy split is used so that crack growth is mainly driven by tensile deformation, not compression.

3.7 Plastic Strain Energy Density

The plastic strain energy density is calculated incrementally as

$$\psi^p = \int_0^T \boldsymbol{\sigma} : \dot{\boldsymbol{\varepsilon}}^p dt.$$

This represents the accumulated plastic work density.

3.8 Fatigue Degradation Function

Fatigue is introduced by reducing the effective fracture toughness through a fatigue degradation function:

$$f(\alpha) = \begin{cases} 1, & \alpha \leq \alpha_T, \\ \left(\frac{2\alpha_T}{\alpha + \alpha_T} \right)^2, & \alpha > \alpha_T. \end{cases}$$

Here:

- α is the accumulated fatigue history variable,
- α_T is the fatigue threshold.

The fatigue threshold is

$$\alpha_T = \frac{G_c}{12l}.$$

3.9 Fatigue History Variable

The fatigue driving quantity is

$$\bar{\alpha} = g(\phi) (\psi_e^+ + \psi^p).$$

The accumulated fatigue history is

$$\alpha = \int_0^T H(\dot{\bar{\alpha}}) |\dot{\bar{\alpha}}| dt.$$

Here, $H(\cdot)$ is the Heaviside function.

This means fatigue history accumulates only when the fatigue driving quantity increases.

3.10 Final Energy Functional with Fatigue

After adding fatigue degradation and elastic energy split, the total energy becomes

$$\begin{aligned}\Psi = & \int_{\Omega} [g(\phi)(\psi_e^+ + \psi^p) + \psi_e^-] dV \\ & + \int_{\Omega} f(\alpha) \frac{G_c}{2l} (\phi^2 + l^2 |\nabla \phi|^2) dV - \int_{\Omega} \mathbf{b} \cdot \mathbf{u} dV - \int_{\partial\Omega} \mathbf{t} \cdot \mathbf{u} dS.\end{aligned}$$

3.11 History Field for Crack Irreversibility

To enforce crack irreversibility, a history field is introduced:

$$P = \max_{\tau \in [0, t]} \{ \psi_e^+ + \eta \psi^p \}.$$

Here, η controls the contribution of plastic work to the crack-driving history field. The paper studies cases such as

$$\eta = 0.1 \quad \text{and} \quad \eta = 1.$$

3.12 Governing Equations

The governing equations are

$$\begin{cases} \nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = 0, \\ g'(\phi)P + \frac{f(\alpha)G_c}{l}\phi - G_c l \nabla \cdot [f(\alpha) \nabla \phi] = 0. \end{cases}$$

The paper uses a hybrid phase-field form:

$$\begin{cases} \nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = 0, \\ g'(\phi)P + \frac{f(\alpha)G_c}{l}\phi - G_c l \nabla \cdot [f(\alpha) \nabla \phi] = 0, \\ \phi = 0 \quad \text{if} \quad \psi_e^+ < \psi_e^-. \end{cases}$$

The hybrid form keeps the displacement equation computationally simpler while still avoiding compression-driven crack growth.

3.13 Plastic Constitutive Models

The paper verifies the method using three plasticity models:

1. linear isotropic hardening,
2. power-law hardening,
3. Armstrong–Frederick cyclic plasticity.

All models are based on the von Mises yield criterion.

3.14 Yield Function

The yield function is

$$F_y = \sigma_{\text{eq}} - Q.$$

Here:

- σ_{eq} is the equivalent von Mises stress,
- Q is the current yield stress.

3.15 Equivalent Stress for Isotropic Hardening

For linear and power-law isotropic hardening models,

$$\sigma_{\text{eq}} = \sqrt{\frac{3}{2} \mathbf{s} : \mathbf{s}},$$

where $\mathbf{s}$ is the deviatoric stress tensor.

3.16 Plastic Flow Rule

The plastic strain rate is

$$\dot{\boldsymbol{\varepsilon}}^p = \sqrt{\frac{3}{2} \dot{\boldsymbol{\varepsilon}}_{\text{eq}}^p} \frac{\mathbf{s}}{\sqrt{\mathbf{s} : \mathbf{s}}}.$$

The equivalent plastic strain rate is

$$\dot{\varepsilon}_{\text{eq}}^p = \sqrt{\frac{2}{3} \dot{\boldsymbol{\varepsilon}}^p : \dot{\boldsymbol{\varepsilon}}^p}.$$

3.17 Yield Stress for Linear and Power-Law Hardening

The current yield stress is

$$Q = \begin{cases} \sigma_y + h \varepsilon_{\text{eq}}^p, & \text{linear isotropic hardening,} \\ \sigma_y \left(1 + \frac{E}{\sigma_y} \varepsilon_{\text{eq}}^p \right)^n, & \text{power-law hardening.} \end{cases}$$

Here:

- σ_y is the initial yield stress,
- h is the linear hardening modulus,
- E is Young's modulus,
- n is the strain hardening exponent.

3.18 Armstrong–Frederick Model

For the Armstrong–Frederick model, the isotropic hardening rule is

$$Q = \sigma_y + (Q_{\text{sat}} - \sigma_y) [1 - \exp(-b \varepsilon_{\text{eq}}^p)].$$

The equivalent stress is

$$\sigma_{\text{eq}} = \sqrt{\frac{3}{2} (\mathbf{s} - \boldsymbol{\theta}) : (\mathbf{s} - \boldsymbol{\theta})}.$$

The plastic strain rate becomes

$$\dot{\boldsymbol{\varepsilon}}^p = \sqrt{\frac{3}{2} \dot{\boldsymbol{\varepsilon}}_{\text{eq}}^p} \frac{\mathbf{s} - \boldsymbol{\theta}}{\sqrt{(\mathbf{s} - \boldsymbol{\theta}) : (\mathbf{s} - \boldsymbol{\theta})}}.$$

The backstress evolution is

$$\dot{\boldsymbol{\theta}} = \frac{2}{3} C \dot{\boldsymbol{\varepsilon}}^p - \gamma \boldsymbol{\theta} \dot{\varepsilon}_{\text{eq}}^p.$$

Here:

- $\boldsymbol{\theta}$ is the backstress tensor,
- C is the kinematic hardening modulus,
- γ is the dynamic recovery parameter.

3.19 Finite Element Discretization

The displacement and phase fields are approximated as

$$\mathbf{u} = \sum_{i=1}^m N_i^u \mathbf{u}_i, \quad \phi = \sum_{i=1}^m N_i^\phi \phi_i.$$

The strain and phase-field gradient are

$$\boldsymbol{\varepsilon} = \sum_{i=1}^m \mathbf{B}_i^u \mathbf{u}_i, \quad \nabla \phi = \sum_{i=1}^m \mathbf{B}_i^\phi \phi_i.$$

3.20 Residual Equations

The displacement residual is

$$\begin{aligned} \mathbf{R}_i^u &= \int_{\Omega} (\mathbf{B}_i^u)^T \boldsymbol{\sigma} dV - \int_{\Omega} (N_i^u)^T \mathbf{b} dV \\ &\quad - \int_{\partial\Omega} (N_i^u)^T \mathbf{t} dS. \end{aligned}$$

The phase-field residual is

$$R_i^\phi = \int_{\Omega} \left\{ \left[\frac{f(\alpha)G_c}{l} \phi + g'(\phi)P \right] N_i^\phi + f(\alpha)G_c l (\mathbf{B}_i^\phi)^T \nabla \phi \right\} dV.$$

3.21 Stiffness Matrices

The displacement stiffness matrix is

$$\mathbf{K}_{ij}^u = \frac{\partial \mathbf{R}_i^u}{\partial \mathbf{u}_j} = \int_{\Omega} g(\phi) (\mathbf{B}_i^u)^T \mathbf{D} \mathbf{B}_j^u dV.$$

The phase-field stiffness matrix is

$$\begin{aligned} K_{ij}^\phi &= \frac{\partial R_i^\phi}{\partial \phi_j} = \int_{\Omega} \left\{ \left[\frac{f(\alpha)G_c}{l} + g''(\phi)P \right] N_i^\phi N_j^\phi \right. \\ &\quad \left. + f(\alpha)G_c l (\mathbf{B}_i^\phi)^T \mathbf{B}_j^\phi \right\} dV. \end{aligned}$$

Here, $\mathbf{D}$ is the consistent tangent stiffness matrix of the intact elasto-plastic material.

3.22 Cycle-Jump Variable Vector

The selected variable vector for cycle jumping is

$$\boldsymbol{\beta} = [P, \alpha, \varepsilon_{\text{eq}}^p].$$

Here:

- P controls the phase-field crack-driving force,
- α controls fatigue degradation,
- $\varepsilon_{\text{eq}}^p$ accounts for plastic strain accumulation at the crack tip.

This is one of the most important choices in the paper.

3.23 Why These Variables Are Selected

The method does not extrapolate every field variable. Instead, it updates the quantities that most directly control crack propagation.

- P determines the history-dependent crack-driving energy.
- α controls the reduction of fracture resistance under fatigue.
- $\varepsilon_{\text{eq}}^p$ accounts for plastic accumulation around the crack tip.

Thus, the crack propagation path and rate can be predicted without explicitly extrapolating all displacement and phase-field variables.

3.24 Evolution Rate of β

The evolution rate of β at cycle N is defined as

$$\mathbf{v}_N = \left. \frac{\partial \beta}{\partial N} \right|_N.$$

In discrete form,

$$\mathbf{v}_N = \beta_N - \beta_{N-1}.$$

This is the cycle-to-cycle change of the variable vector.

3.25 Trapezoidal Estimate of Variable Change

The change in β over a cycle jump ΔN is approximated as

$$\Delta \beta = \Delta N \frac{\mathbf{v}_N + \mathbf{v}_{N+\Delta N}}{2}.$$

Therefore,

$$\beta_{N+\Delta N} = \beta_N + \Delta N \frac{\mathbf{v}_N + \mathbf{v}_{N+\Delta N}}{2}.$$

3.26 Earlier Second-Order Taylor Form

Earlier explicit cycle-jump methods approximate the variable update as

$$\beta_{N+\Delta N} = \beta_N + \Delta N \mathbf{v}_N + \frac{\Delta N^2}{2} \mathbf{a}_N.$$

Here,

$$\mathbf{a}_N = \mathbf{v}_N - \mathbf{v}_{N-1}.$$

This assumes that the evolution rate $\mathbf{v}$ changes linearly during the jump.

3.27 Modified Evolution Rate for Nonlinear Evolution

Li et al. modify the evolution rate to better account for nonlinear evolution:

$$\mathbf{v}_{N+\Delta N} = \mathbf{v}_N + \Delta N \mathbf{a}_N + \frac{\Delta N^2}{2} (\mathbf{a}_N - \mathbf{a}_{N-1}).$$

Substituting this into the trapezoidal expression gives

$$\beta_{N+\Delta N} = \beta_N + \Delta N \mathbf{v}_N + \frac{\Delta N^2}{2} \mathbf{a}_N + \frac{\Delta N^3}{4} (\mathbf{a}_N - \mathbf{a}_{N-1}).$$

3.28 Expression Using Previous Cycle Values

Using

$$\mathbf{v}_N = \beta_N - \beta_{N-1},$$

and

$$\mathbf{a}_N = \mathbf{v}_N - \mathbf{v}_{N-1},$$

the update becomes

$$\begin{aligned}\beta_{N+\Delta N} &= \beta_N + \Delta N(\beta_N - \beta_{N-1}) \\ &\quad + \frac{\Delta N^2}{2}(\beta_N - 2\beta_{N-1} + \beta_{N-2}) \\ &\quad + \frac{\Delta N^3}{4}(\beta_N - 3\beta_{N-1} + 3\beta_{N-2} - \beta_{N-3}).\end{aligned}$$

3.29 Adapted Formula Using Three Complete Cycles

To avoid computing four complete cycles, the paper uses the beginning value of cycle $N - 2$:

$$\begin{aligned}\beta_{q,N+\Delta N} &= \beta_{q,N} + \Delta N(\beta_{q,N} - \beta_{q,N-1}) \\ &\quad + \frac{\Delta N^2}{2}(\beta_{q,N} - 2\beta_{q,N-1} + \beta_{q,N-2}) \\ &\quad + \frac{\Delta N^3}{4}(\beta_{q,N} - 3\beta_{q,N-1} + 3\beta_{q,N-2} - \beta_{0,N-2}).\end{aligned}$$

Here:

- subscript q denotes the end of a cycle,
- subscript 0 denotes the beginning of a cycle,
- $\beta_{q,N}$ is the value at the end of cycle N ,
- $\beta_{0,N-2}$ is the value at the beginning of cycle $N - 2$.

3.30 Incremental Update Form

Because the evolution expression is nonlinear, the jump interval is divided into unit intervals. The final update is written as

$$\beta_{q,N+\Delta N} = \beta_{q,N} + \sum_{i=0}^{\Delta N-1} \Delta \beta_i.$$

The interval increment is

$$\begin{aligned}\Delta \beta_i &= (\beta_{q,N+i} - \beta_{q,N+i-1}) \\ &\quad + \frac{1}{2}(\beta_{q,N+i} - 2\beta_{q,N+i-1} + \beta_{q,N+i-2}) \\ &\quad + \frac{1}{4}(\beta_{q,N+i} - 3\beta_{q,N+i-1} + 3\beta_{q,N+i-2} - \beta_{0,N+i-2}).\end{aligned}$$

This recursive update is used during the explicit cycle-jump step.

3.31 Adaptive Cycle-Jump Size

The cycle-jump size ΔN is selected adaptively based on phase-field evolution.

The phase field after a proposed jump is estimated as

$$\phi_{N+\Delta N} = \phi_N + \Delta N(\phi_N - \phi_{N-1}) + \frac{\Delta N^2}{2}(\phi_N - 2\phi_{N-1} + \phi_{N-2}).$$

The adaptive condition is

$$\Delta\phi_{N+\Delta N} \leq \phi_{\text{cr}}.$$

Here, ϕ_{cr} is the allowable threshold for the phase-field change.

3.32 Threshold for Phase-Field Evolution

The threshold is defined as

$$\phi_{\text{cr}} = \delta_{\text{cr}}\chi_{\text{cr}}\phi.$$

Here:

- δ_{cr} is a correction coefficient related to the phase-field evolution rate,
- χ_{cr} is an upper-limit correction factor,
- ϕ is the current phase-field value.

3.33 Correction Coefficient

The correction coefficient is

$$\delta_{\text{cr},N} = \frac{1}{1 + \left(\frac{\partial^2 \phi}{\partial N^2} \Big|_N \right) / \left(\frac{\partial \phi}{\partial N} \Big|_N \right)}.$$

In discrete form,

$$\delta_{\text{cr},N} = \frac{\phi_N - \phi_{N-1}}{(\phi_N - \phi_{N-1}) + (\phi_N - 2\phi_{N-1} + \phi_{N-2})}.$$

3.34 Final Adaptive Criterion

Combining the phase-field prediction and threshold gives

$$\begin{aligned} & \Delta N(\phi_N - \phi_{N-1}) + \frac{\Delta N^2}{2}(\phi_N - 2\phi_{N-1} + \phi_{N-2}) \\ & \leq \frac{\phi_N - \phi_{N-1}}{(\phi_N - \phi_{N-1}) + (\phi_N - 2\phi_{N-1} + \phi_{N-2})} \chi_{\text{cr}} \phi_N. \end{aligned}$$

This criterion is solved iteratively to obtain the allowable cycle-jump size.

3.35 Element-Wise Jump Size and Global Minimum

Because the adaptive cycle-jump calculation is performed element-wise in the UEL implementation, each element may produce a different jump size.

The element-wise jump size is stored as

$$\Delta N(i, e) = \Delta N_{i,e},$$

where i is the node number and e is the element number.

The global jump size is then chosen as

$$\Delta N_{\min} = \min(\Delta N(i, e)) - 1.$$

This makes the most restrictive crack-tip region control the global jump.

3.36 Complete Adaptive Explicit Cycle-Jump Algorithm

The algorithm is:

1. Compute three complete cycles.

2. Store

$$\beta_{0,N-2}, \quad \beta_{q,N-2}, \quad \beta_{q,N-1}, \quad \beta_{q,N}.$$

3. Compute the phase-field threshold ϕ_{cr} .

4. Initialize trial cycle-jump size $\Delta N = 1$.

5. Estimate $\Delta\phi_{N+\Delta N}$.

6. Increase ΔN while

$$\Delta\phi_{N+\Delta N} \leq \phi_{\text{cr}}.$$

7. Stop if the maximum allowed jump size is reached.

8. Store element-wise jump sizes.

9. Select the global minimum jump size.

10. Update β using the recursive cycle-jump formula.

11. Solve for the displacement and phase fields after the jump.

12. Continue with the next acceleration period.

3.37 Behavior of the Adaptive Jump Size

The adaptive jump size usually has three stages:

1. **Initial stage:** ΔN is small because phase-field evolution near the initial crack tip changes rapidly.
2. **Stable propagation stage:** ΔN becomes large because crack growth is relatively stable.
3. **Final failure stage:** ΔN decreases again because crack propagation accelerates rapidly.

Thus, the method automatically balances accuracy and efficiency.

3.38 Implementation in Abaqus

The method is implemented in Abaqus using:

- UEL for displacement and phase-field equations,
- UMAT with CPE4 elements as a post-processing layer,
- return mapping for the plastic constitutive models,
- staggered solution of displacement and phase field.

3.39 Numerical Validation Cases

The method is verified using several examples:

- single-edge notched tension specimen,
- single-edge notched shear specimen,
- three-point bending specimen,
- three-crack square plate under biaxial loading,
- compact tension specimen.

3.40 Main Numerical Results

The paper shows that the proposed method:

- reproduces fatigue crack paths accurately,
- predicts fatigue crack propagation life with low relative error,
- works for elastic and elasto-plastic materials,
- works for different plasticity models,
- significantly reduces calculation time,
- reproduces Paris-law behavior in the compact tension example.

Most reported relative errors are below approximately 5% for the tested cases.

3.41 Paris Law Verification

For the compact tension specimen, the paper checks whether the accelerated simulation follows Paris' law:

$$\frac{da}{dN} = L(\Delta K)^m.$$

The stress intensity factor range is calculated as

$$\Delta K = \frac{\Delta F}{B\sqrt{W}} \frac{2 + a/W}{(1 - a/W)^{1.5}} \left[0.886 + 4.64 \left(\frac{a}{W} \right) - 13.32 \left(\frac{a}{W} \right)^2 + 14.72 \left(\frac{a}{W} \right)^3 - 5.6 \left(\frac{a}{W} \right)^4 \right].$$

Here:

- a is crack length,
- B is specimen thickness,
- W is specimen width,
- ΔF is the load range.

The paper reports that the accelerated and unaccelerated ΔK - da/dN curves are consistent.

3.42 Key Technical Takeaways

- The method uses an explicit cycle-jump strategy.
- Three complete cycles are computed before each jump.
- The jump variables are

$$\beta = [P, \alpha, \varepsilon_{\text{eq}}^p].$$

- The proposed update formula includes nonlinear evolution of the variable rate.
- The adaptive jump size is controlled by phase-field evolution.
- The global jump size is the minimum value over elements and nodes.
- Crack-tip phase-field evolution naturally controls the jump size.
- The method is suitable for elasto-plastic fatigue crack propagation.

3.43 Limitations

- The method has been verified mainly for classical elasto-plastic constitutive models.
- More complex viscoplastic models need further study.
- Non-proportional multiaxial loading requires further research.
- The choice of crack-driving force and plasticity model can affect the result.
- The adaptive correction factor χ_{cr} influences both accuracy and efficiency.

3.44 Relevance for Thesis Work

This paper is important because it connects cycle jumping with elasto-plastic phase-field fatigue crack propagation.

For thesis work, the most important lessons are:

- cycle-jump methods can be adapted to phase-field fatigue,
- the crack-driving variables should be selected carefully,
- plastic strain accumulation must be included for elasto-plastic fatigue,
- phase-field evolution is a good adaptive criterion for crack-growth simulations,
- using only selected variables can reduce computational cost,
- adaptive jump control is necessary because crack growth is strongly nonlinear near failure.

3.45 One-Sentence Summary

Li et al.'s method accelerates elasto-plastic phase-field fatigue crack propagation by explicitly extrapolating the crack-driving variables P , α , and $\varepsilon_{\text{eq}}^p$ over skipped cycles while adaptively selecting the jump size from the current phase-field evolution.

4 Cheng, Hu and Kirka (2022): Cycle-Jump Acceleration Method for Crystal Plasticity High-Cycle Fatigue

4.1 Full Reference

Cheng, J., Hu, X., and Kirka, M. (2022). *A cycle-jump acceleration method for the crystal plasticity simulation of high cycle fatigue of the metallic microstructure*. International Journal of Fatigue.

4.2 Main Problem Addressed

This paper develops a cycle-jump acceleration method for crystal plasticity finite element simulations of high-cycle fatigue in metallic microstructures.

Direct cycle-by-cycle crystal plasticity finite element simulation is computationally expensive because:

- each loading and unloading cycle requires many finite element increments,
- each integration point contains many crystal plasticity state variables,
- high-cycle fatigue may require thousands or millions of cycles,
- microstructure-level stress redistribution evolves slowly over cycles.

The proposed method accelerates the simulation by extrapolating the internal crystal plasticity variables over skipped cycles and then using a re-equilibration step to recover the displacement and stress fields.

4.3 Main Idea of the Method

The central idea is:

Compute a few complete cycles, estimate the evolution rate of the crystal plasticity internal variables, extrapolate these variables over ΔN skipped cycles, then perform a re-equilibration step so that the finite element displacement and stress fields become consistent with the extrapolated material state.

Unlike some structural cycle-jump methods, this method does not directly extrapolate displacement or stress. Instead, it extrapolates the internal variables that store the material history.

4.4 Microstructure-Based Model

The paper studies an ACMZ aluminium alloy microstructure obtained from EBSD data.

The microstructure contains:

- aluminium matrix grains,
- Cu-based grain-boundary particles,
- grain orientations from EBSD,
- finite element mesh representation of the microstructure.

The aluminium grains are modeled using crystal plasticity, while the Cu-based particles are treated as elastic.

4.5 Crystal Plasticity Kinematics

The total deformation gradient is multiplicatively decomposed into elastic and plastic parts:

$$\mathbf{F} = \mathbf{F}^e \mathbf{F}^p.$$

Here:

- $\mathbf{F}$ is the total deformation gradient,
- $\mathbf{F}^e$ is the elastic deformation gradient,
- $\mathbf{F}^p$ is the plastic deformation gradient.

The plastic velocity gradient is

$$\mathbf{L}^p = \dot{\mathbf{F}}^p (\mathbf{F}^p)^{-1}.$$

For crystal plasticity, it is written as the sum of slip-system contributions:

$$\mathbf{L}^p = \sum_{\alpha} \dot{\gamma}^{\alpha} \mathbf{s}^{\alpha} \otimes \mathbf{m}^{\alpha}.$$

Here:

- α is the slip-system index,
- $\dot{\gamma}^{\alpha}$ is the slip rate on slip system α ,
- $\mathbf{s}^{\alpha}$ is the slip direction,
- $\mathbf{m}^{\alpha}$ is the slip-plane normal.

4.6 Stress Relation

The second Piola–Kirchhoff stress is obtained from the elastic strain measure:

$$\mathbf{S} = \mathbb{C}^e : \mathbf{E}^e.$$

Here:

- $\mathbf{S}$ is the second Piola–Kirchhoff stress,
- $\mathbb{C}^e$ is the anisotropic elastic stiffness tensor,
- $\mathbf{E}^e$ is the elastic Green strain.

The Cauchy stress is obtained by pushing forward the stress measure to the current configuration.

4.7 Power-Law Slip Rule

The slip rate on slip system α is governed by a power-law relation:

$$\dot{\gamma}^{\alpha} = \dot{\gamma}_0 \left| \frac{\tau^{\alpha} - \chi^{\alpha}}{g^{\alpha}} \right|^{1/m} \text{sign}(\tau^{\alpha} - \chi^{\alpha}).$$

Here:

- $\dot{\gamma}_0$ is the reference slip rate,
- τ^{α} is the resolved shear stress,
- χ^{α} is the slip-system backstress,
- g^{α} is the slip resistance,
- m is the strain-rate sensitivity parameter.

4.8 Accumulated Slip

The accumulated slip is

$$\gamma = \int_0^t \sum_{\alpha} |\dot{\gamma}^{\alpha}| d\xi.$$

This variable measures the total accumulated slip activity in the crystal.

4.9 Slip Resistance Hardening

The slip resistance evolves according to a saturation-type hardening law:

$$g^\alpha = g_0^\alpha + (g_s^\alpha - g_0^\alpha) \left[1 - \exp \left(-\frac{h_0^\alpha \gamma}{g_s^\alpha} \right) \right].$$

Here:

- g_0^α is the initial slip resistance,
- g_s^α is the saturation slip resistance,
- h_0^α is the hardening parameter,
- γ is the accumulated slip.

4.10 Hall–Petch Grain-Size Dependence

The initial slip resistance includes grain-size dependence through a Hall–Petch type relation:

$$g_0^\alpha = \bar{g}_0^\alpha + \frac{k}{\sqrt{d^\alpha}}.$$

Here:

- $\bar{g}_0^\alpha$ is the base initial slip resistance,
- k is the Hall–Petch coefficient,
- d^α is the equivalent grain size.

4.11 Armstrong–Frederick Backstress Evolution

The slip-system backstress evolves according to an Armstrong–Frederick type law:

$$\dot{\chi}^\alpha = c\dot{\gamma}^\alpha - d\chi^\alpha |\dot{\gamma}^\alpha|.$$

Here:

- c controls kinematic hardening,
- d controls dynamic recovery,
- χ^α shifts the slip response and stores cyclic hardening memory.

4.12 Internal Variable Set for Cycle Jumping

The crystal plasticity material state is described by the internal variable set

$$\mathcal{S} = \{\mathbf{F}^p, \gamma, \chi^\alpha\}.$$

Here:

- $\mathbf{F}^p$ stores permanent plastic deformation,
- γ stores accumulated slip,
- χ^α stores the slip-system backstress.

This internal variable set is responsible for the path dependence of the crystal plasticity finite element solution.

4.13 Core Principle of the Cycle-Jump Method

The cycle-jump procedure consists of five main steps:

1. Compute a sufficient number of complete cycles.
2. Evaluate the evolution rate of the internal variables at each integration point.
3. Extrapolate the internal variables over ΔN skipped cycles.
4. Replace the internal variables in the UMAT by the extrapolated values.
5. Perform a re-equilibration step to recover consistent displacement and stress fields.

4.14 Two Key Postulations

The method relies on two important assumptions.

Postulation 1

The internal variables evolve slowly and smoothly enough during high-cycle plasticity that they can be extrapolated over a limited number of cycles without significant loss of accuracy.

Postulation 2

If the extrapolated internal variables are accurate, then the displacement and stress fields after the skipped cycles can be recovered accurately by a finite element re-equilibration step.

4.15 Why Re-Equilibration Is Needed

During the skipped cycles, plastic deformation accumulates. The extrapolated internal variables represent this accumulated plastic state, but the total displacement field has not yet been updated.

Therefore, after the jump:

- the plastic strain state is updated,
- the total deformation field is still inconsistent,
- elastic strain and stress are not yet equilibrated.

The re-equilibration step fixes the extrapolated internal variables and lets the implicit finite element solver find a compatible displacement and stress field.

This is why the method is naturally suited to Abaqus/Standard and implicit finite element simulation.

4.16 Taylor Expansion of Internal Variables

At integration point i , the internal variable array after a cycle jump is approximated by a Taylor expansion in cycle space:

$$\mathcal{S}_i^{N+\Delta N} = \mathcal{S}_i^N + \Delta N (\mathcal{S}_i)' + \frac{\Delta N^2}{2} (\mathcal{S}_i)'' + O(\Delta N^3).$$

Here:

- N is the current cycle number,
- ΔN is the number of skipped cycles,
- $\mathcal{S}_i^N$ is the internal state at cycle N ,
- $(\mathcal{S}_i)'$ is the first derivative with respect to cycle number,
- $(\mathcal{S}_i)''$ is the second derivative with respect to cycle number.

4.17 Backward Difference Approximation

The first derivative is approximated by a backward difference:

$$(\mathcal{S}_i)' \approx \mathcal{S}_i^N - \mathcal{S}_i^{N-1}.$$

The second derivative is approximated as

$$(\mathcal{S}_i)'' \approx \mathcal{S}_i^N - 2\mathcal{S}_i^{N-1} + \mathcal{S}_i^{N-2}.$$

Therefore, at least three complete cycles are required before each jump to evaluate the first and second cycle-space derivatives.

4.18 Linear Extrapolation Used in the Paper

Although the Taylor expansion contains higher-order terms, the implemented method uses a backward Eulerian linear approximation:

$$\mathcal{S}_i^{N+\Delta N} = \mathcal{S}_i^N + \Delta N (\mathcal{S}_i)'.$$

This is accurate only if the neglected second-order term is sufficiently small.

4.19 Adaptive Error Criterion

To control the jump length, the second-order term is required to be small compared with the current value of the internal variable:

$$\frac{\Delta N^2}{2} |(\mathcal{S}_i)''| \leq |\mathcal{S}_i^N| \epsilon.$$

Here, ϵ is the accuracy control parameter.

4.20 Allowed Local Jump Size

Solving the above inequality gives the allowable local jump size:

$$\Delta N_i \leq \sqrt{\frac{2|\mathcal{S}_i^N| \epsilon}{|(\mathcal{S}_i)''|}}.$$

This equation is the main adaptive jump-size criterion in the paper.

Interpretation

If the second derivative is small,

$$|(\mathcal{S}_i)''| \rightarrow 0,$$

then the state variable evolves almost linearly and a large jump is allowed.

If the second derivative is large, the material state is changing nonlinearly and the jump size must be reduced.

4.21 Global Cycle-Jump Size

The local allowable jump size is calculated at every integration point.

The global jump size is selected as

$$\Delta N = \min_i [\Delta N_i].$$

An upper bound is also imposed:

$$\Delta N \leq \Delta N_{\max}.$$

Therefore, the most rapidly evolving integration point controls the global cycle jump.

4.22 Complete Cycle-Jump Algorithm

The full algorithm is:

1. Start at cycle N .
2. Compute at least three full cycles explicitly.
3. Store the internal variables

$$\mathcal{S}^N, \quad \mathcal{S}^{N-1}, \quad \mathcal{S}^{N-2}.$$

4. Compute

$$(\mathcal{S})' \quad \text{and} \quad (\mathcal{S})''.$$

5. Calculate the local allowable jump size ΔN_i at each integration point.
6. Select the global value

$$\Delta N = \min_i [\Delta N_i].$$

7. Limit the jump by

$$\Delta N \leq \Delta N_{\max}.$$

8. Extrapolate the internal variables:

$$\mathcal{S}^{N+\Delta N} = \mathcal{S}^N + \Delta N(\mathcal{S})'.$$

9. Replace the UMAT state variables by the extrapolated values.
10. Run the re-equilibration virtual increment.
11. Update the effective cycle number:

$$N \leftarrow N + \Delta N.$$

12. Continue with the next block of cycles.

4.23 Known-State-Variable Cycle-Jump Validation

The paper first validates the method using a known-state-variable cycle-jump test.

In this test:

- a normal cycle-by-cycle reference simulation is first performed,
- internal variables are stored at every cycle,
- a second simulation skips cycles by reading the already-known reference internal variables,
- the result is compared with the reference solution.

This test checks whether accurate internal variables are sufficient to recover the correct displacement and stress fields through re-equilibration.

The result shows that if the internal variables are known accurately, the skipped-cycle displacement and stress fields can be recovered accurately.

4.24 Implementation in Abaqus/Standard

The method is implemented in Abaqus/Standard using user subroutines:

- UEXTERNALDB,
- UMAT,
- DLOAD,
- DISP.

4.25 Role of UEXTERNALDB

The UEXTERNALDB subroutine is used to manage global internal-variable arrays.

Its main tasks are:

- initialize arrays for internal variables at all integration points,
- store internal variables from previous cycles,
- compute first and second cycle-space derivatives,
- determine the jump size ΔN ,
- extrapolate internal variables after the jump,
- update the effective cycle number and effective time.

4.26 Role of UMAT

The UMAT subroutine performs the crystal plasticity constitutive calculation.

During ordinary cycles:

UMAT $\rightarrow$ regular crystal plasticity update.

After a cycle jump:

UMAT $\rightarrow$ read extrapolated internal variables.

Then the re-equilibration step is performed by fixing the extrapolated state variables and computing stress and tangent stiffness for virtual increments.

4.27 Role of DLOAD and DISP

For load-controlled cyclic plasticity:

- DLOAD applies periodic loading based on effective time,
- during re-equilibration, zero load increment is applied.

For displacement-controlled cyclic plasticity:

- DISP applies periodic displacement based on effective time,
- during re-equilibration, zero displacement increment is applied.

4.28 Effective Cycle Number and Virtual Time

The paper distinguishes between:

- Abaqus step time,
- effective cycle number,
- effective simulation time.

After a jump,

$$N_{\text{eff}} \leftarrow N_{\text{eff}} + \Delta N.$$

The re-equilibration increment is called a virtual increment because it is not a physical loading increment. It is only used to restore equilibrium after updating the internal material state.

4.29 Acceleration Behavior

The jump size is small during early cycles because the material state evolves rapidly during initial plastic redistribution.

As the material response stabilizes, the internal variables evolve more smoothly and the allowable jump size increases.

Therefore:

$$\text{rapid evolution} \Rightarrow \Delta N \text{ small,}$$

$$\text{stable evolution} \Rightarrow \Delta N \text{ large.}$$

4.30 Validation Cases

The paper validates the method under several conditions:

- load-controlled cyclic plasticity with peak stress of 120 MPa and $R = 0$,
- higher load magnitude with peak stress of 150 MPa,
- negative load ratio case with $R = -0.8$,
- strain-controlled cyclic plasticity,
- elastic shakedown condition,
- dislocation-density-based crystal plasticity with SSD and GND.

4.31 Important Numerical Findings

The paper reports the following trends:

- At lower load, the acceleration efficiency is very high because the internal variables evolve smoothly after initial stabilization.
- At higher load, the stress and internal variables evolve faster, so the acceleration efficiency is lower.
- A negative R ratio causes stronger nonlinear kinematic hardening and reduces the acceleration efficiency.
- Strain-controlled cyclic plasticity can also be accelerated successfully.
- Elastic shakedown reduces acceleration efficiency during the transition stage, but the acceleration rate recovers after the microstructure becomes mostly elastic.
- Dislocation-density-based crystal plasticity can also be accelerated, although GND effects may reduce efficiency because they create localized deformation near grain boundaries.

4.32 Accuracy Measures

The paper assesses accuracy by comparing:

- displacement evolution,
- von Mises stress distribution,
- accumulated slip evolution,
- mean absolute error in the stress field.

For the main validation cases, the accelerated results show good agreement with the cycle-by-cycle reference solution.

4.33 Key Technical Takeaways

- The method accelerates CPFE high-cycle fatigue simulations by extrapolating internal variables, not external response quantities.
- The internal variables used are

$$\mathcal{S} = \{\mathbf{F}^p, \gamma, \chi^\alpha\}.$$

- A backward Eulerian linear extrapolation is used.
- The jump size is controlled by the second cycle-space derivative of the internal variables.
- The global jump size is the minimum over all integration points.
- Re-equilibration is essential after each jump.
- The method is well suited for Abaqus/Standard because re-equilibration requires an implicit solver.

4.34 Limitations

- The method assumes that internal variables evolve smoothly enough over skipped cycles.
- Strongly nonlinear evolution reduces the allowable jump size.
- Reversing an inaccurate jump is difficult in commercial Abaqus.
- The method is currently tied to implicit finite element simulation.
- More complex dislocation-density or higher-order GND models may reduce acceleration efficiency.

4.35 Relevance for Thesis Work

This paper is highly relevant for cycle-jump implementation in user material subroutines because it clearly shows how cycle jumping can be performed at the internal-variable level.

For thesis work, the most important lessons are:

- do not extrapolate only stress or displacement;
- extrapolate the material history variables stored inside the constitutive model;
- use previous cycles to compute cycle-space derivatives;
- choose the jump size based on an error criterion;
- after jumping, perform re-equilibration to recover a consistent stress and displacement field;
- use global arrays or external databases to manage state variables across cycles.

4.36 One-Sentence Summary

Cheng, Hu and Kirka’s method accelerates high-cycle crystal plasticity fatigue simulations by extrapolating the internal variables $\mathbf{F}^p$, accumulated slip γ , and slip-system backstress χ^α over skipped cycles, then performing an implicit re-equilibration step in Abaqus/Standard to recover the consistent displacement and stress fields.

5 Comparative Summary of Cycle-Jump Techniques

Comparison Point	Cojocaru and Karlsson (2006)	Nesnas and Saanouni (2000)	Li et al. (2025)	Cheng, Hu and Kirka (2022)
Main objective	Accelerate finite element simulations of cyclically loaded structures with slowly evolving structural or material properties.	Accelerate numerical integration of coupled damage-viscoplastic constitutive equations under cyclic loading.	Accelerate elasto-plastic phase-field fatigue crack propagation simulations.	Accelerate crystal plasticity finite element simulations for high-cycle fatigue of metallic microstructures.
Level of application	Structural finite element level.	Constitutive/material-point level and structural finite element level.	Phase-field finite element level using UEL/UMAT implementation.	Crystal plasticity finite element level using UMAT and global state-variable storage.
Main physical problem	Cyclic structural evolution, especially thermal barrier coating evolution, transformation strain, oxidation-growth type problems, and slowly changing material properties.	Low-cycle fatigue with coupled viscoplasticity and damage evolution.	Fatigue crack propagation in elastic and elasto-plastic materials using phase-field fracture.	High-cycle fatigue in metallic microstructures where grain orientation, slip, particles, and local stress redistribution are important.
Variables used for cycle jump	General finite element variables such as displacement, stress, strain, and possibly equivalent plastic strain. In the examples, displacement components u_1 and u_2 are mainly used as control variables.	Representative internal variables from the asymptotic constitutive integration, mainly ΔL and damage D .	Selected phase-field and plasticity variables: $\beta = [P, \alpha, \varepsilon_{eq}^p]$. Here, P is the crack-driving history field, α is the fatigue history variable, and ε_{eq}^p is equivalent plastic strain.	Crystal plasticity internal variables: $\mathbf{S} = \{\mathbf{F}^p, \gamma, \chi^\alpha\}$. Here, $\mathbf{F}^p$ is plastic deformation gradient, γ is accumulated slip, and χ^α is slip-system backstress.
Type of extrapolation	Heun-type extrapolation using the current and predicted evolution slopes.	Taylor expansion in cycle space. Two versions are discussed: direct extrapolation of all state variables or extrapolation of selected scalar variables followed by reconstruction.	Explicit nonlinear cycle-jump extrapolation. The method modifies the evolution-rate prediction to include nonlinear evolution of β .	Backward Eulerian linear extrapolation of crystal plasticity internal variables.
Basic evolution-rate idea	Uses slopes between characteristic values from consecutive cycles: $s_{12} = \frac{y_1 - y_2}{\Delta t_{\text{cycle}}}, \quad s_{23} = \frac{y_2 - y_3}{\Delta t_{\text{cycle}}}.$	Uses pseudo-derivatives of state variables with respect to cycle number: $\dot{\mathbf{y}}_N \approx \mathbf{y}_{N+1} - \mathbf{y}_N.$	Uses cycle-to-cycle evolution rate: $\mathbf{v}_N = \beta_N - \beta_{N-1}.$ Also uses acceleration of this evolution rate.	Uses first and second backward differences of internal variables: $(\mathbf{S})' \approx \mathbf{S}^N - \mathbf{S}^{N-1},$ $(\mathbf{S})'' \approx \mathbf{S}^N - 2\mathbf{S}^{N-1} + \mathbf{S}^{N-2}.$
Jump-size control	Jump size is controlled by limiting the change in predicted slope: $\Delta t_{y,jump}^M = q_y \Delta t_{\text{cycle}} \frac{ s_{12} }{ s_{12} - s_{23} }.$	Jump size is controlled mainly by ΔL and D : $\Delta N = \min(\Delta N_{\Delta L}, \Delta N_D).$	Jump size is adaptively controlled by phase-field evolution: $\Delta \phi_{N+\Delta N} \leq \phi_{cr}.$ The most restrictive element controls the global jump.	Jump size is controlled by requiring the second-order term to remain small: $\frac{\Delta N^2}{2} (\mathbf{S}_i)' \leq \mathbf{S}_i^N \epsilon.$
Global jump selection	Minimum admissible jump over all selected variables and material points/nodes.	Minimum jump over selected control variables and integration points.	Minimum jump over all elements/nodes: $\Delta N_{\min} = \min(\Delta N(i, e)) - 1.$	Minimum admissible jump over all integration points: $\Delta N = \min_i [\Delta N_i].$

6 Next Paper

References

- [1] D. Cojocaru and A. M. Karlsson, “A simple numerical method of cycle jumps for cyclically loaded structures,” *International Journal of Fatigue*, vol. 28, pp. 1677–1689, 2006.
- [2] J. Cheng, X. Hu, and M. Kirka, “A cycle-jump acceleration method for the crystal plasticity simulation of high cycle fatigue of the metallic microstructure,” *International Journal of Fatigue*, 2022.